

Monte Carlo Option Pricing: Exercises Summary

Suman Mishra

September 16, 2025

Introduction

The price dynamic of an underlying asset is often modeled by a **Geometric Brownian Motion** (GBM), given by the stochastic differential equation:

$$dS_t = \mu S_t dt + \sigma S_t dW_t,$$

where S_t is the asset price at time t and W_t is a standard Brownian motion. Under the *risk-neutral measure*, μ is replaced by the risk-free interest rate r . The solution to this SDE is

$$S_T = S_0 \exp \left(\left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right),$$

where $Z \sim \mathcal{N}(0, 1)$ is a standard normal random variable.

To price European call and put options with maturity T , Monte Carlo simulation generates many random samples of S_T according to the GBM model, computes the option payoffs for each simulated terminal price, discounts them back to present value, and averages over all simulations. The Monte Carlo estimator converges to the true option price as the number of simulations n increases.

The algorithm to estimate V_0 by Monte-Carlo simulation is :

1. Simulate a set of sample values $S_T^{(1)}, S_T^{(2)}, \dots, S_T^{(n)}$ for S_T . We simulate this sample by generating the standard normal random variable ϵ using the inverse transform method.
2. For each $S_T^{(i)}$, we discount back:

$$Y_i = e^{-rT} V_T \left(S_T^{(i)} \right).$$

3. Using the law of large numbers, we estimate V_0 by

$$V_0 = E^Q \left[e^{-rT} V_T (S_T) \right] \approx \frac{Y_1 + Y_2 + \dots + Y_n}{n}.$$

This report summarizes two exercises investigating the accuracy and behavior of Monte Carlo pricing for European options under GBM dynamics.

Summary of Exercises

Exercise 1: Accuracy vs Number of Simulations

In this exercise, we studied the relationship between the number of Monte Carlo simulations n and the accuracy of option price estimates. The key parameters were:

- Underlying asset price $S_0 = 60$,
- Strike price $K = 65$,
- Time to maturity $T = 1$ year,
- Risk-free rate $r = 0.03$,
- Volatility $\sigma = 0.3$.

Observations:

- As the number of simulation trials n increases, the Monte Carlo estimates for both call and put option prices become more precise.
- The percent standard error (%SE) decreases significantly with larger n , showing that increasing the number of simulations improves accuracy.
- Runtime generally increases with n , but remains reasonable even for large numbers of trials (up to 500,000).

Exercise 2: Impact of Extreme Strike Prices on Monte Carlo Accuracy

This exercise explored how the strike price K influences Monte Carlo pricing accuracy. We fixed the underlying asset price at $S_0 = 60$, time to maturity $T = 1$, risk-free rate $r = 0.03$, and volatility $\sigma = 0.3$. Strike prices were varied from 20 to 120.

Monte Carlo simulations with $n = 5,000,000$ trials were used to estimate call and put prices, and we computed the percent standard error.

Observations:

- The percent standard error (%SE) for call options increases as the strike price K moves further away from the current stock price S_0 , especially for higher values of K . This indicates greater uncertainty in estimating large out-of-the-money call option prices.
- On the other hand, the %SE for put options decreases with increasing K , as put options become more valuable and their price estimates become more stable.
- For very low strike prices K (large out-of-the-money put options), the %SE can be very high or undefined because the put option price approaches zero, making relative error calculations unstable.

Exercise 1 Python Code

```
1 import numpy as np
2 import time
3
4 # params.
5 S0 = 60
6 K = 65
7 T = 1.0
8 r = 0.03
9 sigma = 0.3
10
11 # trials
12 ns = [10000, 20000, 50000, 100000, 200000, 500000]
13
14 # Function to simulate stock prices  $S_T$ 
15 def simulate_ST(S0, r, sigma, T, n):
16     epsilon = np.random.normal(0, 1, n)
17     ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) *
18                     epsilon)
19     return ST
20
21 # Function to calculate option prices with % standard error and
22 # runtime
23 def monte_carlo(S0, K, r, sigma, T, n):
24     start_time = time.time()
25     ST = simulate_ST(S0, r, sigma, T, n)
26     payoff_call = np.maximum(ST - K, 0)
27     payoff_put = np.maximum(K - ST, 0)
28     discount_call = np.exp(-r * T) * payoff_call
29     discount_put = np.exp(-r * T) * payoff_put
30     price_call = np.mean(discount_call)
31     price_put = np.mean(discount_put)
32     se_call = np.std(discount_call, ddof=1) / np.sqrt(n)
33     se_put = np.std(discount_put, ddof=1) / np.sqrt(n)
34     pct_se_call = (se_call / price_call) * 100
35     pct_se_put = (se_put / price_put) * 100
36     runtime = time.time() - start_time
37     return n, price_call, price_put, pct_se_call, pct_se_put,
38         runtime
39
40 results = []
41 for n in ns:
42     results.append(monte_carlo(S0, K, r, sigma, T, n))
43
44 # Output
45 output = []
```

```

43 output.append(f"{'n':>10} {'Call Price':>12} {'Put Price':>12} {'%SE  

    Call':>12} {'%SE Put':>12} {'Runtime (s)':>12}")
44 for row in results:
45     output.append(f"{row[0]:10d} {row[1]:12.4f} {row[2]:12.4f} {row  

    [3]:12.4f} {row[4]:12.4f} {row[5]:12.4f}")
46
47 output_str = '\n'.join(output)
48 print(output_str)

```

Exercise 1 Output

n	Call Price	Put Price	%SE Call	%SE Put	Runtime (s)
10000	5.9762	8.9316	1.9263	1.1136	0.0027
20000	6.0122	8.9716	1.3815	0.7851	0.0022
50000	5.9974	8.9670	0.8809	0.4949	0.0030
100000	5.9156	8.9623	0.6197	0.3497	0.0047
200000	5.8569	9.0026	0.4418	0.2467	0.0095
500000	5.8929	8.9795	0.2786	0.1561	0.0424

Table 1: Monte Carlo simulation results for option prices and percent standard errors at different number of trials n .

Exercise 2 Python Code

```

1  # params
2  S0 = 60
3  T = 1.0
4  r = 0.03
5  sigma = 0.3
6  n = 500000  # Use fixed n
7
8  # Range of strike prices K
9  Ks = np.arange(20, 121, 10)
10
11 # Function to simulate terminal stock prices  $S_T$ 
12 def simulate_ST(S0, r, sigma, T, n):
13     epsilon = np.random.normal(0, 1, n)
14     ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) *
        epsilon)
15     return ST
16
17 # Function to calculate percent standard error for call and put for
    changing strike K

```

```

18 def monte_carlo(S0, K, r, sigma, T, n):
19     ST = simulate_ST(S0, r, sigma, T, n)
20     payoff_call = np.maximum(ST - K, 0)
21     payoff_put = np.maximum(K - ST, 0)
22     discount_call = np.exp(-r * T) * payoff_call
23     discount_put = np.exp(-r * T) * payoff_put
24     price_call = np.mean(discount_call)
25     price_put = np.mean(discount_put)
26     se_call = np.std(discount_call, ddof=1) / np.sqrt(n)
27     se_put = np.std(discount_put, ddof=1) / np.sqrt(n)
28     pct_se_call = (se_call / price_call) * 100 if price_call > 0
        else 0
29     pct_se_put = (se_put / price_put) * 100 if price_put > 0 else 0
30     return K, pct_se_call, pct_se_put
31
32 results = []
33 for K in Ks:
34     results.append(monte_carlo(S0, K, r, sigma, T, n))
35
36 # Prepare output table
37 output = []
38 output.append(f"{'K':>10} {'%SE Call':>12} {'%SE Put':>12}")
39 for row in results:
40     output.append(f"{row[0]:10d} {row[1]:12.4f} {row[2]:12.4f}")
41
42 output_str = '\n'.join(output)
43 print(output_str)

```

Exercise 2 Output

K	%SE Call	%SE Put
20	0.0640	16.7315
30	0.0840	1.6893
40	0.1159	0.5628
50	0.1640	0.2938
60	0.2331	0.1872
70	0.3318	0.1328
80	0.4748	0.1000
90	0.6850	0.0786
100	0.9846	0.0637
110	1.4359	0.0530
120	2.1112	0.0450

Table 2: Percent standard errors for call and put options for different strike prices K .

Possible Questions for Next Class

1. How does time to maturity T influence the accuracy and variance of Monte Carlo price estimates?
2. What does it mean in the reference book that "One major advantage of the Monte Carlo method is that the error is independent of the dimension of the option problem." What do they mean by dimension.